

CSCE 625 ARTIFICIAL INTELLIGENCE · MODULE 09 · LECTURE NOTES

Probabilistic Reasoning

Saying 'usually' properly.

MODULE OUTCOMES

- Apply Bayes' rule and explain base rates.
- Explain conditional independence and why it makes inference possible.
- Build a Bayesian network and read independence off its structure.
- Perform exact inference by variable elimination.
- Choose an approximate method when exact inference is infeasible.

1 Bayes and independence

$$P(H \mid E) = P(E \mid H) \cdot P(H) / P(E)$$

Posterior equals likelihood times prior, normalised. $P(H)$ is the prior and it is not optional — dropping it is the base-rate fallacy, and it is the single most common error in applied probability, including in published work. A test that is 99% accurate in both directions, applied to a condition affecting 1 in 10,000 people, has a positive predictive value of about 1%: of 10,000 people, one has the condition and is probably detected, while about 100 healthy people test positive. **The arithmetic is trivial and the intuition is terrible**, which is why it is worth computing by hand once rather than accepting on authority — and why CSCE 633 Module 12 §2's insistence on reporting precision rather than accuracy matters so much on imbalanced problems. **It is the same arithmetic.**

CONDITIONAL INDEPENDENCE IS WHAT MAKES ANY OF THIS TRACTABLE

A full joint distribution over n binary variables requires $2^n - 1$ numbers. At $n = 30$ that is over a billion parameters — **nobody can specify them, store them, or estimate them from data.**

But most variables are conditionally independent given a few others. Once you know whether it rained, whether the sprinkler ran tells you nothing further about whether the grass is wet having already accounted for both — and **that kind of structure is ubiquitous in real domains.**

So the joint factorises into a product of small conditional distributions, one per variable given its few parents, **and the parameter count drops from exponential in n to linear in n** (with the constant set by the largest parent set).

That is the entire reason probabilistic AI is possible. And it is the same structural argument as Module 06 §3's treewidth and CSCE 669 Module 11's sparsity: structure, not size, decides tractability — which at the third appearance should be read as one of this program's load-bearing ideas rather than three separate facts.

2 Bayesian networks

A DIRECTED ACYCLIC GRAPH. Each node is a variable; each node carries the conditional distribution $P(\text{node} \mid \text{parents})$.

THE JOINT FACTORISES:

$$P(x_1..x_n) = \text{product over } i \text{ of } P(x_i \mid \text{parents}(x_i))$$

SO THE GRAPH IS A CLAIM ABOUT INDEPENDENCE, and THE MISSING EDGES CARRY THE INFORMATION. A fully connected network asserts nothing and costs as much as the full joint.

READING INDEPENDENCE OFF THE GRAPH (d-separation):

CHAIN $A \rightarrow B \rightarrow C$

A and C are independent GIVEN B. Knowing the intermediate blocks the path.

FORK $A \leftarrow B \rightarrow C$

A and C are independent GIVEN B. A common cause explains the correlation.

COLLIDER $A \rightarrow B \leftarrow C$

A and C are independent -- UNTIL you observe B, and THEN THEY BECOME DEPENDENT.

THE COLLIDER IS THE COUNTERINTUITIVE CASE AND IT MATTERS: two unrelated causes of one observed effect become correlated once the effect is known. Your car will not start; the cause is the battery or the fuel pump; learning that the battery is fine makes the fuel pump more likely.

THIS IS EXPLAINING AWAY. It is why observing MORE can CREATE dependence rather than remove it, and it is the mechanism behind selection bias and Berkson's paradox.

Explaining away is the one to hold onto. It is the mechanism behind a great many apparent paradoxes in conditional reasoning — including selection bias, where conditioning on being in a sample (a collider) induces correlations between characteristics that are independent in the population. **And it is a reason to draw the graph before reasoning:** whether conditioning on a variable helps or hurts depends on its position, and that is not something intuition reliably gets right.

3 Inference

VARIABLE ELIMINATION, AND WHAT DECIDES ITS COST

To compute $P(\text{query} \mid \text{evidence})$, sum out each variable that is neither the query nor evidence, one at a time — multiplying together all the factors mentioning that variable, then summing it away to produce a new factor.

The cost is exponential in the size of the largest intermediate factor produced, and the elimination order determines how large that gets — a good order keeps factors small, a bad one produces a factor over most of the network.

Finding the optimal order is NP-hard, and cheap heuristics (min-fill, min-degree) are usually adequate — the same situation as Module 03's heuristics, applied to the ordering rather than to the search.

And the best achievable cost is exponential in the network's treewidth — exactly Module 06 §3's quantity, in a different subject. Which is why that result was worth stating twice: constraint satisfaction and probabilistic inference are both instances of a general computation over a factorised structure, and the same graph property governs both.

Method	How it works	When to use it
Variable elimination	Sum out variables in a heuristically chosen order.	Exact. Viable when the treewidth is low, which many real networks are.
Likelihood weighting	Sample from the network with evidence fixed, weighting each sample by the evidence's likelihood.	Simple to implement; degrades badly when there is a lot of evidence, because almost all weight concentrates on a few samples.
Gibbs sampling / MCMC	Repeatedly resample one variable from its conditional given all the others.	General and widely applicable; convergence is hard to verify — see the note below.
Loopy belief propagation	Pass messages as if the graph were a tree, ignoring cycles.	Fast and approximate, and it may fail to converge or converge to a wrong answer — and it works surprisingly well on many real networks anyway.
Variational inference	Fit a tractable distribution to approximate the intractable posterior, by optimisation.	Fast, biased, and the bias is characterisable — and it is CSCE 669's optimisation applied to inference.

Report which method you used and whether it converged. MCMC that has not mixed produces confident nonsense and reports nothing about it — the chain returns samples, the samples have a mean, and the mean is wrong. Run multiple chains from different starting points and compare them; report the diagnostic. Which is CSCE 753 Module 12's theme arriving in a different subject: a confident output is not a correct one, and the only defence is a check that the method itself does not perform.

4 Probabilistic reasoning in an agent

- **Tracking what the agent believes about hidden state** — where the player probably is, given a sound heard fifteen seconds ago, a last confirmed sighting, and the connectivity of the level. **A belief distribution over locations, updated on each observation**, which is both principled and produces good search behaviour.
- **Sensor fusion** — combining multiple noisy sources of evidence about one quantity, which is **CSCE 753 Module 07's Kalman filter as a special case** (a Bayes net with Gaussian conditionals and linear dynamics).
- **Diagnosis and reasoning about causes**, where §2's explaining away is genuinely needed — fault diagnosis, medical decision support, and root-cause analysis all have collider structure.
- **And decision-making under uncertainty**, which is Module 10 — **beliefs are only useful if they change what the agent does**, and a belief distribution with no decision attached is an expensive way to produce no behaviour.
- **Not for most game AI, honestly. A designer-authored approximation — a visibility cone, a decaying 'last known position' marker, a simple alertness counter — is usually cheaper, far more controllable, and indistinguishable to the player. Probabilistic reasoning earns its place where the uncertainty is real and consequential** (robotics, diagnosis, a stealth game whose whole design rests on modelling what enemies believe) **and not elsewhere** — which is Module 01 §1's cheapest-sufficient-design argument applied to this module's own content.

WHAT THIS BUYS OVER LOGIC

Logic cannot say 'usually' (Module 08 §4), and patching a logical rule base with exceptions is open-ended — each exception needs a rule, and the exceptions have exceptions.

Probability says it natively, degrades gracefully as evidence accumulates rather than flipping between certainties, and combines independent evidence correctly without any new rules — which is the structural advantage, not merely a notational one.

The cost is that you must supply numbers, and the numbers are frequently guesses dressed as measurements. This is a real objection and the right answer is not confidence but **sensitivity analysis**: vary the numbers you are least sure of and report whether the conclusion changes (CSCE 669 Module 12 §3's dual-variable check, in a probabilistic setting).

And exact inference is intractable in general (§3), so in practice you are approximating — which §3's footnote says to report rather than to assume away. **The honest position is that probability is the right language for uncertainty and that using it well requires stating both the numbers you assumed and the approximation you computed with.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Russell & Norvig — AIMA, chapters 12–14 (pseudocode free)	Bayes nets, d-separation, and variable elimination — the reference for the whole module.
Koller & Friedman — Probabilistic Graphical Models	The comprehensive treatment , strong on the treewidth connection of §3. Library copy; the author's free Coursera lectures cover much of it.
Pearl — Causality, chapter 1; and The Book of Why	Where §2's d-separation and explaining away come from , and the causal reading of the graph that the standard treatment understates.
pgmpy documentation (free)	A working library for §2 and §3 , with every inference method in the table — after implementing variable elimination yourself once.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Compute the 1-in-10,000 test example by hand** and confirm the 1% figure.
- 2 **Plot positive predictive value against prevalence** for a fixed test accuracy.
- 3 **Count the parameters** of a full joint over 20 binary variables and of a Bayes net with at most 3 parents each.
- 4 **Build a Bayes net** for a domain you know, with at least eight variables.
- 5 **Identify a chain, a fork, and a collider** in it and state the independence each implies.
- 6 **Demonstrate explaining away numerically** on your collider.
- 7 **Implement variable elimination** and verify against enumeration on a small network.
- 8 **Compare two elimination orders** and report the largest intermediate factor for each.
- 9 **Implement Gibbs sampling** and run four chains. Report whether they agree.
- 10 **Deliberately run MCMC too briefly** and report the wrong answer it gives confidently.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State Bayes' rule and explain the base-rate fallacy with numbers.
- 2 Why does conditional independence make probabilistic AI possible?
- 3 What does a Bayes net's structure assert, and what do missing edges mean?
- 4 Describe chain, fork, and collider, and which is counterintuitive.
- 5 Explain explaining away and name a paradox it accounts for.

- 6 How does variable elimination work and what decides its cost?
- 7 What quantity governs it, and where else in this program does it appear?
- 8 Name five inference methods and when each applies.
- 9 Why must you report convergence?
- 10 What does probability buy over logic, and what does it cost?